

R documentation

of ‘man/CircadianAnalysisOptions.Rd’ etc.

July 13, 2026

CircadianAnalysisOptions

Create Analysis & Reporting Options

Description

Create Analysis & Reporting Options

Usage

```
CircadianAnalysisOptions(  
  alpha = 0.05,  
  p.adjust.method = "BH",  
  parallel.ncores = 1,  
  amp.cutoff = 0,  
  target_effect = 0.1,  
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),  
  reference_n = 60,  
  r_strata = c(0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 4.5, 5),  
  strata_labels = NULL,  
  phase_shifts = c(0, 0.5, 1, 2, 4, 6, 8, 10, 12),  
  DCmethod = "DCP"  
)
```

Arguments

alpha	Significance level for DCP pipeline
p.adjust.method	Multiple testing correction method
parallel.ncores	Number of cores for parallel DCP
amp.cutoff	Amplitude cutoff for DCP_Rhythmicity
target_effect	Minimum effect size to count as "interesting"
fdr_thresholds	FDR thresholds for power curves

reference_n	Reference sample size for Panel B / phase shift plots
r_strata	Breakpoints for A/sigma stratification
strata_labels	Labels for strata (auto-generated if NULL)
phase_shifts	Phase shift magnitudes to sweep (for sensitivity analysis)
DCmethod	Differential-rhythmicity detector to report against, one of "DCP" (default) or "CircaCompare".

Value

Object of class "CircadianAnalysisOptions"

CircadianBioOptions

Create Biology + Differential Options

Description

Create Biology + Differential Options

Usage

```
CircadianBioOptions(
  ngenes = 5000,
  prop_rhythmic = NULL,
  period = 24,
  lBaselineExpr = NULL,
  lBaselineExpr2 = NULL,
  lod = NULL,
  lod2 = NULL,
  amplitude = NULL,
  amplitude2 = NULL,
  sigma_rhythmic = NULL,
  paired_sigma = FALSE,
  paired_omega = FALSE,
  paired_alpha = FALSE,
  omega_rhythmic = NULL,
  alpha_rhythmic = NULL,
  omega_dist = NULL,
  alpha_dist = NULL,
  cts = NULL,
  cts2 = NULL,
  phase = "uniform",
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  mesor_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
```

```

    dr_amp_scale = 1,
    dr_sigma_scale = 1,
    alpha_pilot = NULL,
    sim.seed = 12345
)

```

Arguments

<code>ngenes</code>	Number of genes.
<code>prop_rhythmic</code>	Proportion of rhythmic genes. If <code>NULL</code> and a pilot dataset name is supplied to <code>lBaselineExpr</code> , the pilot's estimated proportion is used.
<code>period</code>	Circadian period in hours.
<code>lBaselineExpr</code>	Log baseline expression. Can be a numeric scalar (constant for all genes), a numeric vector (resampled to <code>ngenes</code>), a function <code>f(n)</code> returning <code>n</code> values, or a character string naming a built-in pilot dataset stored in <code>data/</code> . No default, must be supplied.
<code>lBaselineExpr2</code>	As <code>lBaselineExpr</code> but for group 2 (differential analyses only). <code>NULL</code> uses the same distribution as group 1.
<code>lOD</code>	Log over-dispersion (noise). Same forms accepted as <code>lBaselineExpr</code> . No default, must be supplied.
<code>lOD2</code>	As <code>lOD</code> but for group 2. <code>NULL</code> shares group 1 values.
<code>amplitude</code>	Amplitude distribution for rhythmic genes. Same forms accepted as <code>lBaselineExpr</code> . No default, must be supplied.
<code>amplitude2</code>	As <code>amplitude</code> but for group 2.
<code>sigma_rhythmic</code>	Optional numeric vector of per-gene noise values for rhythmic genes (same length as <code>amplitude</code>). When provided, <code>amplitude</code> and <code>sigma</code> are drawn jointly to preserve pilot A-sigma correlation.
<code>paired_sigma</code>	Logical. If <code>TRUE</code> and <code>sigma_rhythmic</code> is supplied, expand <code>amplitude</code> and <code>sigma_rhythmic</code> jointly by the same resampled gene index so their pilot-estimated pairwise correlation is preserved; if <code>FALSE</code> (default), <code>amplitude</code> and <code>sigma</code> are expanded independently.
<code>paired_omega</code>	Logical. If <code>TRUE</code> and <code>omega_rhythmic</code> is supplied (same length as <code>amplitude</code>), expand <code>omega</code> using the same shared gene-index used for <code>amplitude/sigma</code> so all three are paired per gene.
<code>paired_alpha</code>	Logical. Same as <code>paired_omega</code> for <code>alpha_rhythmic</code> .
<code>omega_rhythmic</code>	Optional numeric vector of per-gene FMM omega values from a pilot FMM fit (same length as <code>amplitude</code>). Used when <code>paired_omega = TRUE</code> ; ignored when <code>omega_dist</code> is also set.
<code>alpha_rhythmic</code>	Optional numeric vector of per-gene FMM alpha values in radians from a pilot FMM fit (same length as <code>amplitude</code>).
<code>omega_dist</code>	Optional list specifying a distribution from which omega is drawn at simulation time, e.g. <code>list(family="beta", a=1, b=5)</code> or <code>list(family="fixed", value=0.7)</code> . Overrides <code>omega_rhythmic</code> when both are provided.

<code>alpha_dist</code>	Optional list specifying a distribution for alpha, <code>list(family="normal", mean=0, sd_hours=2)</code> . Internally <code>sd_hours</code> is converted to radians via <code>sd_rad = sd_hours * 2*pi/24</code> . When <code>alpha_rhythmic</code> is also provided and lengths match, the perturbation is added to each gene's empirical alpha (<code>alpha_g = alpha_rhythmic[g] + N(0, sd_rad^2)</code>). When <code>alpha_rhythmic</code> is missing, <code>alpha_g = N(0, sd_rad^2)</code> .
<code>cts</code>	Numeric vector of sample collection times for passive design.
<code>cts2</code>	As <code>cts</code> for group 2.
<code>phase</code>	Phase distribution for rhythmic genes. "uniform" (default) or the same forms as <code>lBaselineExpr</code> .
<code>prop_DR</code>	Proportion with differential rhythmicity.
<code>prop_DP</code>	Proportion with differential phase.
<code>prop_DM</code>	Proportion with differential mesor (mean shift, both groups rhythmic).
<code>phase_diff</code>	Range of phase shift for DP genes <code>c(min, max)</code> .
<code>amp_diff</code>	Range of amplitude ratio (unused; retained for interface compatibility).
<code>mesor_diff</code>	Range of mesor shift for DM genes <code>c(min, max)</code> (additive, log-scale units).
<code>dp_shift_mode</code>	"fixed" (use <code>phase_diff[2]</code>) or "uniform" (sample uniformly within <code>phase_diff</code> range).
<code>dr_amp_scale</code>	Scale factor for amplitude (A) to adjust DR strength.
<code>dr_sigma_scale</code>	Scale factor for sigma to adjust DR strength.
<code>alpha_pilot</code>	Rhythmicity pre-screen threshold used to define the pilot's rhythmic gene set (the value at which <code>prop_rhythmic</code> and the effect-size distribution were calibrated). Stored on the object for provenance and reporting; default <code>NULL</code> .
<code>sim.seed</code>	Random seed.

Value

Object of class "CircadianBioOptions".

CircadianBootstrapOptions

Create Bootstrap Design Grid Options

Description

Create Bootstrap Design Grid Options

Usage

```
CircadianBootstrapOptions(
  design_vector,
  B_values = c(4, 6, 8, 12, 24),
  m_values = NULL,
  N_values = NULL,
  nboot = 200,
```

```

    nsims_inner = 20,
    design = c("active", "passive"),
    seed = 42
  )

```

Arguments

<code>design_vector</code>	Ordered time points to sweep over (length $\geq \max(B_values)$)
<code>B_values</code>	Number of distinct time points to test
<code>m_values</code>	Replicates per time point (if NULL, derived as $\text{round}(N/B)$)
<code>N_values</code>	Total per-group sample sizes to test (if NULL, derived as $B*m$)
<code>nboot</code>	Number of bootstrap parameter draws (outer uncertainty loop)
<code>nsims_inner</code>	Simulations per bootstrap draw
<code>design</code>	"active" or "passive"
<code>seed</code>	Random seed

Value

Object of class "CircadianBootstrapOptions"

```

CircadianDesignOptions
  Create Study Design Options

```

Description

Create Study Design Options

Usage

```

CircadianDesignOptions(
  sample_sizes = c(10, 20, 40, 60, 80, 100),
  nsims = 100,
  design = c("active", "passive"),
  cts = NULL,
  B_values = NULL,
  test_types = c("DR", "DP", "DM"),
  omega = 1,
  beta = pi
)

```

Arguments

<code>sample_sizes</code>	Vector of sample sizes per group
<code>nsims</code>	Number of simulation replicates
<code>design</code>	"active" or "passive"
<code>cts</code>	Time-of-day distribution for passive design (numeric vector)

B_values	Optional vector of time-point counts (B) to sweep, in addition to sample_sizes; NULL runs a single B implied by cts.
test_types	Character vector of tests to run ("DR", "DP", "DM")
omega	FMM waveform shape parameter in (0, 1]. omega = 1 (default) is the pure cosinor path; omega < 1 simulates a non-sinusoidal waveform via simCircadianFMM(). omega = 0 is not allowed.
beta	FMM orientation parameter (peak-location offset, default pi); only used when omega < 1, ignored for the cosinor path.

Value

Object of class "CircadianDesignOptions"

`circaPowerApproxN80`

Estimate n for 80 percent power using CircaPower at median pilot r

Description

Estimate n for 80 percent power using CircaPower at median pilot r

Usage

```
circaPowerApproxN80(
  bio.opts,
  alpha = 0.05,
  target_power = 0.8,
  n_search = seq(5, 500, by = 5)
)
```

Arguments

bio.opts	CircadianBioOptions with amplitude and sigma_rhythmic.
alpha	Significance level (default 0.05).
target_power	Target power (default 0.80).
n_search	Integer vector of candidate sample sizes.

Value

Single integer – smallest n achieving target_power, or NA if not found.

detect_cosinor	<i>Unified cosinor rhythmicity detection (single- or multi-harmonic)</i>
----------------	--

Description

Canonical entry point for rhythmicity detection by K-harmonic cosinor regression. $K = 1$ is the single-harmonic cosinor F-test, identical to `detect_DCP`; $K = 2$ adds the 12-hour second harmonic and tests the joint harmonic block. By default the unshrunk exact nested F-test is used (matching the power derivations in the paper). Returns raw p-values; the caller applies FDR control.

Usage

```
detect_cosinor(expr, times, K = 1L, period = 24, ebayes = FALSE, mc.cores = 1L)
```

Arguments

<code>expr</code>	Gene x sample expression matrix
<code>times</code>	Numeric time vector (length = <code>ncol(expr)</code>)
<code>K</code>	Harmonic order (default 1)
<code>period</code>	Period in hours (default 24)
<code>ebayes</code>	For $K \geq 2$, moderate residual variance with empirical-Bayes shrinkage (default FALSE, the unshrunk exact F-test)
<code>mc.cores</code>	Cores for the $K \geq 2$ fit (default 1)

Value

Numeric vector of raw p-values, length `nrow(expr)`

See Also

[detect_DCP](#)

<code>estCircadianParam</code>	<i>Estimate Circadian Parameters from Pilot Data (Single-Cohort)</i>
--------------------------------	--

Description

Fits a cosinor model per gene, ranks rhythmic genes by p-value, and returns empirical parameter distributions packaged as a `CircadianBioOptions` object suitable for downstream simulation and power analysis.

Usage

```
estCircadianParam(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data</code>	Numeric matrix of pilot expression (genes x samples). A <code>data.frame</code> is coerced to matrix.
<code>times</code>	Numeric vector of sample collection times in hours, length <code>ncol(data)</code> . The function does NOT validate units; pass hours (0 to 24 scale).
<code>period</code>	Period in the same units as <code>times</code> (default 24 hours).
<code>min_rhythm_pval</code>	Uncorrected cosinor p-value threshold for the rhythmic-candidate set (default 0.01). Used to compute <code>prop_rhythmic</code> and to select the top-K ($K = \min(\text{top_k}, n_{\text{rhythmic}})$) genes whose amplitudes, phases, and noise drive simulation truth.
<code>top_k</code>	Number of highest-signal genes (ranked by cosinor p-value) that form the effect-size distribution (amplitude, phase, noise). Default 300. If <code>top_k</code> exceeds the number of rhythmic genes, it is capped to that number with a warning. <code>Inf</code> or <code>"all"</code> uses every rhythmic candidate.
<code>prop_DR, prop_DP, prop_DM</code>	Proportions of differentially-rhythmic, differentially-phased, and differentially-mesor genes (defaults 0.15, 0.10, 0.00). Capped automatically at <code>prop_rhythmic</code> .
<code>mesor_diff, phase_diff, amp_diff</code>	Numeric length-2 ranges for the differential effect-size sampling.
<code>dp_shift_mode</code>	One of <code>"fixed"</code> or <code>"uniform"</code> (default <code>"fixed"</code>).
<code>dr_amp_scale, dr_sigma_scale</code>	Scaling factors for DR-class amplitude and noise distributions (default 1).
<code>paired_sigma</code>	Logical, default <code>FALSE</code> . If <code>TRUE</code> , joint (A, sigma) sampling preserves pilot correlation in downstream simulations.
<code>sim.seed</code>	Integer seed for downstream simulation (default 12345).
<code>verbose</code>	Logical, default <code>TRUE</code> . Print summary at end.

Value

A CircadianBioOptions S3 object usable directly with `runSimsSingleCohort()` or `runDifferentialPower()`.

See Also

`estCircadianParamFMM` for FMM-aware pre-screen, `estCircadianParamTwoGroup` for two-group setup, `prepCircadianData` for normalisation upstream.

Examples

```
## Not run:
set.seed(1)
n_genes <- 200; n_samples <- 30
times <- rep(seq(0, 22, by = 2), length.out = n_samples)
expr <- matrix(rnorm(n_genes * n_samples), nrow = n_genes)
for (g in 1:20) expr[g, ] <- expr[g, ] + 2 * cos(2*pi*times/24)
bio <- estCircadianParam(expr, times, period = 24, verbose = FALSE)

## End(Not run)
```

estCircadianParam2H

Estimate CircadianBioOptions Under the Two-Harmonic Cosinor Model

Description

Two-harmonic analog of `estCircadianParam`. Fits the regression

$$y = \mu + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) + a_2 \cos(2\omega_0 t) + b_2 \sin(2\omega_0 t) + \epsilon$$

to each gene by OLS, tests the joint rhythmicity null $H_0 : a_1 = b_1 = a_2 = b_2 = 0$ on 4 d.f., and returns a CircadianBioOptions object whose 5-way joint empirical distribution $F_{(\mu, A_1, \phi_1, A_2, \phi_2, \sigma)}$ is preserved by gene-paired index draws downstream.

The returned object carries the standard 1-harmonic fields (`amplitude`, `phase`, `sigma_rhythmic`) holding the 1st-harmonic $A_{g,1}$, $\phi_{g,1}$, and σ_g respectively, and three extra fields attached to the list after construction:

- `amplitude2`: 2nd-harmonic amplitudes $A_{g,2}$
- `phase2`: 2nd-harmonic acrophases $\phi_{g,2}$ in hours $[0, period/2)$
- `paired_2h`: TRUE flag (downstream simulators read this to switch on the 2-harmonic generative model).

These extras live alongside `$diagnostics`; the constructor is not modified.

Usage

```
estCircadianParam2H(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 500,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data</code>	Numeric matrix of pilot expression (genes x samples).
<code>times</code>	Numeric vector of sample collection times in hours, length <code>ncol(data)</code> .
<code>period</code>	Period in the same units as <code>times</code> (default 24 hours).
<code>min_rhythm_pval</code>	P-value threshold for the joint 4-d.f. F-test (default 0.01). Genes with $p < \text{this}$ define the candidate set.
<code>top_k</code>	Cap on number of genes contributing to the joint $(A_1, \phi_1, A_2, \phi_2, \sigma)$ distribution (default 500), ranked by F-test p-value.
<code>prop_DR, prop_DP, prop_DM</code>	Differential proportions (defaults 0.15, 0.10, 0.00). Forwarded unchanged to <code>CircadianBioOptions</code> ; note the K=2 extension itself is single-cohort only.
<code>mesor_diff, phase_diff, amp_diff</code>	Differential effect-size ranges.
<code>dp_shift_mode</code>	"fixed" or "uniform" (default "fixed").
<code>dr_amp_scale, dr_sigma_scale</code>	DR scale factors (default 1).
<code>paired_sigma</code>	Logical (default FALSE). Forwarded to <code>CircadianBioOptions</code> .
<code>sim.seed</code>	Integer seed (default 12345).
<code>verbose</code>	Logical (default TRUE). Print diagnostic summary.

Value

A `CircadianBioOptions` object with extra fields `$amplitude2`, `$phase2`, `$paired_2h`, and a `$diagnostics` list summarising the 2H fit (median A2/A1 ratio, fraction of genes with significant 2nd-harmonic block, etc.).

Methodological notes

1. Pilot summary is a 5-way joint empirical distribution. When the simulator draws a rhythmic gene, the same index j is used across $(A_1, \phi_1, A_2, \phi_2, \sigma)$, preserving the cross-parameter dependence observed in the pilot.
2. No 2nd-harmonic significance threshold is applied: ALL top-K genes passing the joint 4-d.f. F-test contribute, even those whose A_2 is essentially noise. The empirical F_{A_2} will have a noise spike near zero, which is fine.
3. K=2 extension is single-cohort only – the differential simulator (`simCircadianDiff`) is not extended.

See Also

`estCircadianParam` for the 1-harmonic case, `simCircadianSingleCohort2H` for the matching simulator.

```
estCircadianParamTwoGroup
```

Estimate a two-group circadian pilot for differential power

Description

Fits cosinor models to two groups of pilot data and assembles the two-group pilot summary used by the differential power simulator, encoding differential rhythmicity (DR), differential phase (DP), and differential MESOR (DM) structure.

Usage

```
estCircadianParamTwoGroup(
  data_1,
  data_2,
  times_1,
  times_2,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  phase_shift_threshold = 2,
  prop_DM = NULL,
  mesor_diff = NULL,
  dp_shift_mode = c("uniform", "fixed"),
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

`data_1, data_2`

Gene-by-sample expression matrices for the two groups.

`times_1, times_2`

Numeric collection times for each group.

<code>period</code>	Period in hours (default 24).
<code>min_rhythm_pval</code>	Marginal p-value screen defining the rhythmic pilot set.
<code>top_k</code>	Number of highest-signal genes (per group, ranked by cosinor p-value) forming each group's effect-size distribution (default 300). Capped (with a warning) to a group's rhythmic-gene count if larger; <code>Inf</code> or <code>"all"</code> uses every rhythmic candidate.
<code>phase_shift_threshold</code>	Minimum phase difference (hours) for the DP class.
<code>prop_DM</code>	Optional target proportion of differential-MESOR genes.
<code>mesor_diff</code>	Optional MESOR difference for the DM component.
<code>dp_shift_mode</code>	Differential-phase shift model, <code>"uniform"</code> or <code>"fixed"</code> .
<code>paired_sigma</code>	Preserve the paired (amplitude, noise) relationship across groups.
<code>sim.seed</code>	Random seed.
<code>verbose</code>	Print progress.

Value

A two-group `CircadianBioOptions` pilot summary.

<code>launchShiny</code>	<i>Launch the SCP Shiny app</i>
--------------------------	---------------------------------

Description

Opens the bundled simulation-based power-analysis app in the user's default browser. The app lets the user pick a bundled pilot (or upload their own), set target power and FDR, read off the recommended sample size, explore per-gene rhythmicity, and run pathway enrichment.

Usage

```
launchShiny(..., install_deps = TRUE, species_all = FALSE)
```

Arguments

<code>...</code>	Additional arguments passed to <code>shiny::runApp()</code> .
<code>install_deps</code>	Logical; if <code>TRUE</code> (default) install any missing optional app packages before launching. Set <code>FALSE</code> to skip the check (missing features then degrade gracefully inside the app).
<code>species_all</code>	Logical; if <code>TRUE</code> also install the annotation databases for rat, zebrafish, <i>Drosophila</i> , and <i>Arabidopsis</i> (in addition to human + mouse) so upload gene-symbol mapping works for every supported species. Default <code>FALSE</code> installs only human + mouse (the common case); the others install on demand or you can pass <code>TRUE</code> once.

Details

On first launch the function checks for the app's optional helper packages and offers to install any that are missing (so the gene-symbol mapping, XLSX export, full-app capture, and pathway-enrichment features work out of the box). These are kept out of the package's hard dependencies so a plain `install.packages("SCP")` stays lightweight and the API works without the large Bioconductor annotation databases.

Value

Invisibly returns `NULL`. Called for its side effect of launching the Shiny app.

Examples

```
## Not run:
  launchShiny()
  launchShiny(species_all = TRUE)      # also install rat/zebrafish/fly/plant DBs
  launchShiny(install_deps = FALSE)    # skip the dependency check

## End(Not run)
```

```
makeAdaptiveRStrata
```

Compute adaptive r-strata breakpoints with fixed bin width

Description

Generates breakpoints from 0 to $\text{ceiling}(r_{\text{max}} / \text{bin_width}) * \text{bin_width}$, stepping by `bin_width`, then appends `Inf`. Every bin width is identical, but the number of bins adapts to the empirical `r` range of the pilot data.

Usage

```
makeAdaptiveRStrata(
  bio.opts,
  bin_width = 0.25,
  r_min_pct = 0,
  r_pctile_cap = 0.99
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions with <code>amplitude</code> and <code>sigma_rhythmic</code> .
<code>bin_width</code>	Width of each <code>r</code> bin (default 0.25).
<code>r_min_pct</code>	Lower percentile of pilot <code>r</code> used as range floor (default 0 = always start from 0).
<code>r_pctile_cap</code>	Upper percentile of pilot <code>r</code> used as the range ceiling before rounding up to the nearest <code>bin_width</code> (default 0.99).

Value

Numeric vector of breakpoints starting at 0 and ending at `Inf`.

npower

*Find the sample size needed to achieve a target genome-wide power***Description**

Works on output from either `runSimsSingleCohort()` or `runSimsDiff()`. Recomputes marginal power at the requested FDR from raw simulation output, then returns the smallest grid `N` that meets the target (`n_grid`) and, when `interpolate = TRUE` (the default), a linearly interpolated estimate between the two bracketing grid points (`n`).

Usage

```
npower(
  res,
  target_power = 0.8,
  fdr = 0.05,
  endpoint = NULL,
  p.adjust.method = "BH",
  interpolate = TRUE
)
```

Arguments

<code>res</code>	Output from <code>runSimsSingleCohort()</code> or <code>runSimsDiff()</code> .
<code>target_power</code>	Numeric in (0, 1). Target marginal power (default 0.80).
<code>fdr</code>	FDR threshold (default 0.05).
<code>endpoint</code>	For <code>runSimsDiff()</code> output: one of "DR", "DP", "DM". Ignored for single-cohort output.
<code>p.adjust.method</code>	Multiple-testing correction (default "BH"). Only applied for single-cohort output (differential uses pre-computed FDR arrays).
<code>interpolate</code>	Logical. If <code>TRUE</code> (default), linearly interpolates between the two adjacent grid points to return a more precise <code>n</code> estimate. Set to <code>FALSE</code> to revert to the grid-snapped <code>n_grid</code> value (backward-compatible).

Value

An object of class "npower" (a list) with:

<code>n</code>	Recommended sample size: <code>n_interp</code> when <code>interpolate = TRUE</code> , otherwise <code>n_grid</code> . NA if target power is never reached.
<code>n_grid</code>	Smallest sample size in the simulation grid achieving target power, or NA.
<code>n_interp</code>	Linearly interpolated sample size between the two bracketing grid points (a fractional <code>N</code> rounded up). NA when <code>interpolate = FALSE</code> or when target power is never reached.
<code>power</code>	Named numeric vector: mean power at each grid sample size.
<code>target_power, fdr, endpoint</code>	Echo of inputs.

plotBootstrapDesignGrid

Plot bootstrap design grid results

Description

Panel A: Power vs N, one line per B, with bootstrap CI ribbon. Panel B: Heatmap of mean power over (N, B) grid.

Usage

```
plotBootstrapDesignGrid(
  result,
  test_type = "DR",
  fdr_threshold = NULL,
  panels = "A",
  output_file = NULL
)
```

Arguments

result	Output from runBootstrapDesignGrid()
test_type	Test type to plot
fdr_threshold	FDR threshold (label only)
panels	Which panel(s) to draw: "A" (power vs N) and/or "B" (heatmap) (default "A").
output_file	Path for PDF output (NULL = screen)

plotDiffPower

Plot differential circadian power

Description

Default layout is marginal power vs sample size only (one column per endpoint, one row per comparison). The effect-size-stratified panels (power and true-discoveries by r) are opt-in via stratified = TRUE, which restores the original 3-column-per-row layout.

Usage

```
plotDiffPower(
  res_list,
  comp_labels = names(res_list),
  endpoints = c("DR", "DP", "DM"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  display_sizes = NULL,
```

```

    stratified = FALSE,
    r_break_width = 0.25,
    r_max = 5,
    r_display_max = NULL,
    main_title = "Differential Circadian Power Analysis",
    out_pdf = NULL,
    width = NULL,
    height = NULL
  )

```

Arguments

<code>res_list</code>	Named list of length 2, each element output of <code>runSimsDiff()</code> .
<code>comp_labels</code>	Character(2), display labels for the two comparisons.
<code>endpoints</code>	Which endpoints to include (default <code>c("DR","DP","DM")</code>).
<code>fdr_thresholds</code>	FDR levels for panel A (default <code>c(0.01,0.05,0.10,0.20)</code>).
<code>panel_fdr</code>	FDR threshold used to draw the marginal power curve in the non-stratified layout (default 0.05).
<code>vline_power</code>	Horizontal reference line for target power, drawn at this value (default 0.80).
<code>vline_fdr</code>	FDR threshold marked with a reference guide on the power panels (default 0.20).
<code>display_sizes</code>	Sample sizes to show in all panels (NULL = all).
<code>stratified</code>	Logical (default FALSE). If FALSE, draws only the marginal power-vs-sample-size panel per endpoint (1 x n_endpoints layout per comparison). If TRUE, also draws power-by-r and true-discoveries-by-r strata panels in the original 3-column layout.
<code>r_break_width</code>	Width of r-strata bins (default 0.25). Ignored if <code>stratified = FALSE</code> .
<code>r_max</code>	Upper bound (in $r = A/\sigma$ units) for the r-strata breakpoints when <code>stratified = TRUE</code> (default 5).
<code>r_display_max</code>	Optional upper x-axis limit (r units) for the stratified panels; NULL (default) uses the full stratified range.
<code>main_title</code>	Outer plot title (default "Differential Circadian Power Analysis").
<code>out_pdf</code>	Path for PDF. If NULL, plots to current device.
<code>width</code>	PDF width in inches. Defaults adapt to stratified.
<code>height</code>	PDF height in inches. Defaults adapt to stratified.

Value

Invisibly, list of panel data per comparison/endpoint.

plotSingleCohortPower

Plot single-cohort circadian power (3 panels)

Description

Plot single-cohort circadian power (3 panels)

Usage

```
plotSingleCohortPower(
  res,
  out_pdf = NULL,
  title = "",
  panel_a_res = NULL,
  panels = c("A", "B", "C"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  fdr = NULL,
  p.adjust.method = "BH",
  reference_n = NULL,
  display_sizes = NULL,
  r_max = 5,
  cex_main = 1.5,
  cex_lab = 1.6,
  cex_axis = 1.45,
  line_lwd = 2,
  pt_cex = 0.95,
  legend_cex = NULL,
  title_cex = 1.65,
  oma_top = 2.4,
  legend_pos = "topleft",
  se_cap = 0.3,
  vertical = FALSE,
  panel_c_legend = FALSE,
  width = 15,
  height = 5.5
)
```

Arguments

res	Output from <code>runSimsSingleCohort()</code> .
out_pdf	Path for PDF output. If NULL, plots to current device.
title	Overall figure title (default empty).
panel_a_res	Optional alternate result object used for the Panel A marginal-power computation only. If NULL (default), Panel A is computed from <code>res</code> . Useful when the marginal summary should reflect a different empirical assumption than the effect-size-stratified panels.

<code>panels</code>	Character vector, which of panels "A" (marginal power vs n), "B" (true discoveries by r-stratum), "C" (stratified power by r-stratum) to draw (default all three).
<code>fdr_thresholds</code>	FDR levels for panel A (default <code>c(0.01,0.05,0.10,0.20)</code>).
<code>panel_fdr</code>	FDR threshold highlighted in panels B/C (default 0.05).
<code>vline_power</code>	Horizontal reference line for target power (default 0.80).
<code>vline_fdr</code>	FDR threshold marked with a reference guide (default 0.20).
<code>fdr</code>	Single-value alias that sets both <code>panel_fdr</code> and <code>vline_fdr</code> at once (back-compatible convenience; default NULL).
<code>p.adjust.method</code>	Correction method (default "BH").
<code>reference_n</code>	Reference sample size for annotations (default NULL = none).
<code>display_sizes</code>	Sample sizes to show in all panels (NULL = all).
<code>r_max</code>	Upper bound ($r = A/\sigma$ units) for the r-strata breakpoints; NULL adapts to the 95th percentile of observed r (default 5).
<code>cex_main</code>	Title character expansion (default 1.50).
<code>cex_lab</code>	Axis-label character expansion (default 1.60).
<code>cex_axis</code>	Axis-tick character expansion (default 1.45).
<code>line_lwd</code>	Line width for power curves (default 2).
<code>pt_cex</code>	Point character expansion for plotted markers (default 0.95).
<code>legend_cex</code>	Legend text character expansion; NULL (default) picks a size automatically.
<code>title_cex</code>	Outer-title character expansion (default 1.65).
<code>oma_top</code>	Outer margin (lines) reserved above the panels for the title (default 2.4).
<code>legend_pos</code>	Legend position keyword passed to <code>legend()</code> (default "topleft").
<code>se_cap</code>	Maximum half-width (power units) drawn for standard-error bars, to keep long bars from overplotting neighboring panels (default 0.3).
<code>vertical</code>	Logical; if TRUE, stack panels vertically instead of the default horizontal 1 x <code>length(panels)</code> layout (default FALSE).
<code>panel_c_legend</code>	Logical; if TRUE, also draw the legend on panel C (default FALSE; panel A's legend is normally sufficient).
<code>width</code>	PDF width in inches (default 15).
<code>height</code>	PDF height in inches (default 5.5).

Value

Invisibly returns list of data used per panel.

```
prepCircadianData
```

Prepare expression data for circadian power analysis

Description

Converts raw expression data into the standard (matrix, times) format expected by `estCircadianParam()`, `estCircadianParamTwoGroup()`, and `runBootstrapDesignGrid()`.

The pipeline always expects:

- data, numeric matrix, **genes x samples**, log2-scale
- times, numeric vector, hours in [0, 24), same length as `ncol(data)`

This function handles the three most common raw formats:

"counts" Raw integer counts (e.g. `featureCounts`, `STAR`). Converts to $\log_2(\text{CPM} + 1)$: `log2(sweep(x, 2, colSums(x), "/") * 1e6 + 1)`.

"cpm" Raw CPM values (e.g. `edgeR cpm()`, baboon data). Converts to $\log_2(\text{CPM} + 1)$.

"log2" Already log2-scale (e.g. `GSE54651`, `Seney ACC`). Wraps with `data.matrix()` only; no numeric transformation.

Usage

```
prepCircadianData(
  expr,
  times,
  input_type = c("counts", "cpm", "log2"),
  pheno = NULL,
  sample_col = NULL,
  time_col = NULL,
  period = 24,
  verbose = TRUE
)
```

Arguments

<code>expr</code>	A matrix, data.frame, or file path (CSV/TSV) of expression values. Rows = genes, columns = samples. Row names used as gene IDs.
<code>times</code>	Numeric vector of time-of-day values (hours, 0-24) for each sample, OR a character column name in <code>pheno</code> to extract times from.
<code>input_type</code>	One of "counts", "cpm", or "log2". Default "counts".
<code>pheno</code>	Optional data.frame of sample metadata. Must have the same column order as <code>expr</code> if <code>times</code> is a column name string.
<code>sample_col</code>	Optional: name of the column in <code>pheno</code> whose values match <code>colnames(expr)</code> for alignment. If NULL, assumes columns are already in the same order as rows of <code>pheno</code> .
<code>time_col</code>	Name of the TOD column in <code>pheno</code> (used when <code>times</code> is a column name string).
<code>period</code>	Circadian period in hours. Default 24. Used only for validation.
<code>verbose</code>	Print a short summary of what was done. Default TRUE.

Value

A list with:

data	Numeric matrix (genes x samples), log2-scale
times	Numeric vector of hours (0-24), length == ncol(data)
n_genes	Number of genes
n_samples	Number of samples
input_type	The input_type used

Examples

```
## Not run:
# From raw counts matrix
prep <- prepCircadianData(counts_matrix, times = tod_vector, input_type = "counts")
bio <- estCircadianParam(prep$data, prep$times)

# From data.frame with metadata
prep <- prepCircadianData(
  expr      = expr_df,
  times     = "TOD",
  input_type = "log2",
  pheno     = pheno_df,
  sample_col = "SampleID"
)

# Baboon raw CPM (load gives a data.frame)
prep <- prepCircadianData(baboon_df, times = tod_vector, input_type = "cpm")

## End(Not run)
```

recommendDesign	<i>B vs m design study – analytical + simulation</i>
-----------------	--

Description

Full B vs m study orchestrator. Three sequential steps:

Usage

```
recommendDesign(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = c("DCP", "JTK", "RAIN", "MH"),
  target_power = 0.8,
  mode = c("single", "differential"),
  run_simulation = TRUE,
  prior_result = NULL,
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
```

```

    plot = TRUE,
    output_file = NULL,
    verbose = TRUE
  )

```

Arguments

<code>bio.opts</code>	CircadianBioOptions from <code>estCircadianParam()</code> or <code>estCircadianParamTwoGroup()</code>
<code>design.opts</code>	CircadianDesignOptions with <code>sample_sizes</code> and <code>B_values</code>
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>methods</code>	Detection methods to compare. Single-cohort: <code>c("DCP", "JTK", "RAIN", "MH")</code> . Differential: <code>c("DCP", "CircaCompare", "LimoRhyde", "DODR")</code> .
<code>target_power</code>	Target power for recommendation (default 0.80)
<code>mode</code>	"single" or "differential" (default "single")
<code>run_simulation</code>	TRUE = run full simulation after analytical step. FALSE = analytical (CircaPower) only.
<code>prior_result</code>	A previous <code>runSingleCohortGrid()/runDifferentialPower()</code> result to reuse instead of re-running simulation.
<code>alpha2</code>	2nd-harmonic deviation (scalar or vector)
<code>alpha3</code>	3rd-harmonic deviation (scalar or vector)
<code>mc.cores</code>	Parallel cores for simulation step
<code>plot</code>	Auto-plot final recommendation
<code>output_file</code>	PDF path (NULL = screen)
<code>verbose</code>	Print progress

Details

1. Print method guidance table (`printMethodGuidance`) 2. Analytical: CircaPower closed-form power at each N (DCP; B-invariant). For JTK/RAIN/MH no closed form exists – simulation is required. 3. Simulation: run `runSingleCohortPower()` or `runDifferentialPower()`, or absorb a `prior_result` from a previous call to skip re-running.

Value

SCPRecommendResult with: `$guidance` – data.frame: method guidance table `$analytical_df` – data.frame[method, N, power_analytical] (DCP only) `$simulation` – `runSingleCohortGrid()` or `runDifferentialPower()` result (NULL if not run) `$recommendation` – data.frame[method, optimal_B, n_target, note]

```
runBootstrapDesignGrid
```

Run bootstrap design grid simulation

Description

Estimates uncertainty in power across (N, B) design configurations by bootstrapping parameter sets from pilot data.

Usage

```
runBootstrapDesignGrid(
  pilot_data,
  pilot_times,
  boot.opts,
  analysis.opts,
  bio_diff.opts,
  pilot_data_2 = NULL,
  pilot_times_2 = NULL,
  mode = NULL,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  min_rhythm_pval = 0.01,
  resample = c("subject", "gene"),
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

<code>pilot_data</code>	Genes x samples matrix (pilot experiment).
<code>pilot_times</code>	Numeric vector of sample time points, length = <code>ncol(pilot_data)</code> .
<code>boot.opts</code>	<code>CircadianBootstrapOptions</code> from <code>CircadianBootstrapOptions()</code> . Must set <code>B_values</code> to a single value for passive designs.
<code>analysis.opts</code>	<code>CircadianAnalysisOptions</code> .
<code>bio_diff.opts</code>	<code>CircadianBioOptions</code> from <code>estCircadianParam()</code> . Required even in <code>mode = "single"</code> , differential proportion fields are read from here.
<code>pilot_data_2</code>	Optional group-2 pilot matrix for <code>mode = "differential"</code> .
<code>pilot_times_2</code>	Optional group-2 pilot times.
<code>mode</code>	"single" or "differential".
<code>methods</code>	Detection method(s): "DCP" (default), "JTK", "RAIN".
<code>test_types</code>	Endpoints for differential mode: any of "DR", "DP", "DM".
<code>alpha2</code>	Second-harmonic coefficient (default 0).

<code>alpha3</code>	Third-harmonic coefficient (default 0).
<code>min_rhythm_pval</code>	P-value threshold for pilot rhythmicity (default 0.01).
<code>resample</code>	Resampling scheme for the outer bootstrap. "subject" (default) is the Efron nonparametric bootstrap over pilot subjects: each draw resamples subject columns with replacement (carrying their collection times) and refits the pilot summary, matching the Section 2.3 description. "gene" is the legacy scheme that fits once and resamples gene rows of the fit; retained for backward compatibility and diagnostics only.
<code>verbose</code>	Print progress (default TRUE).
<code>mc.cores</code>	Parallel cores for inner simulation loop (default 1).

Value

Named list with 15 elements:

<code>N_values</code>	Candidate sample sizes tested.
<code>B_values</code>	Candidate time-point counts tested.
<code>m_matrix</code>	Matrix of total samples (N x B).
<code>nboot</code>	Number of bootstrap replicates.
<code>design</code>	Design type ("active" or "passive").
<code>test_types</code>	Endpoints evaluated.
<code>fdr_threshold</code>	FDR threshold used.
<code>boot_power</code>	4-D array [nboot x n_N x n_B x n_tests].
<code>power_mean</code>	Mean power array [n_N x n_B x n_tests].
<code>power_se</code>	Bootstrap SE array, same shape as <code>power_mean</code> .
<code>power_ci_lo</code>	2.5th percentile CI array.
<code>power_ci_hi</code>	97.5th percentile CI array.
<code>optimal_B</code>	Integer vector (length n_N): B with highest mean power at each N.
<code>optimal_B_ci_lo</code>	Bootstrap 2.5th percentile of optimal B.
<code>optimal_B_ci_hi</code>	Bootstrap 97.5th percentile of optimal B.

```
runDifferentialPower
```

Run two-group differential circadian power analysis

Description

Sweeps N x alpha2 x alpha3 x method x test_type and returns a tidy power table. Automatically calls `plot()` and `npower()` unless `plot=FALSE`.

Usage

```
runDifferentialPower(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

bio.opts	CircadianBioOptions from estCircadianParamTwoGroup()
design.opts	CircadianDesignOptions (sample_sizes, design, cts)
analysis.opts	CircadianAnalysisOptions
methods	Any of "DCP", "CircaCompare"
test_types	Any of "DR", "DP", "DM" (silently NA if method lacks support)
alpha2	Scalar or vector swept for both groups
alpha3	Scalar or vector swept for both groups
mc.cores	Parallel cores
plot	Auto-plot on completion
output_file	PDF path (NULL = screen)
verbose	Print progress

Value

List (class SCPDiffResult) from runSimsDiff(): pval_DR, fdr_DR, pval_DP, fdr_DP, pval_DM, fdr_DM – 3-D arrays [ngenex n_sizes x nsims]; **plus** diff_type, effectsize, sample_sizes, nsims. Pass to plotDiffPower() or npower(..., endpoint="DR").

runSimsSingleCohort

Simulation-Based Power for Single-Cohort Rhythmicity Detection

Description

Extends the closed-form CircaPower approach by running full simulations: for each sample size and simulation replicate, generates data from the empirical pilot parameter distributions, applies the cosinor F-test per gene, applies BH correction, and aggregates empirical power and FDR.

Usage

```
runSimsSingleCohort (
  bio.opts,
  design.opts,
  analysis.opts,
  method = "cosinor",
  K = 1L,
  harmonics = c(0, 0),
  verbose = TRUE,
  mc.cores = 1L,
  progress = NULL
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions – pilot parameter distributions. Build with <code>estCircadianParam()</code> .
<code>design.opts</code>	CircadianDesignOptions – sample sizes, nsims, design.
<code>analysis.opts</code>	CircadianAnalysisOptions – alpha, p.adjust.method.
<code>method</code>	Detection method: "DCP" (1-harmonic cosinor F-test), "JTK", "RAIN", "MH" (multi-harmonic), or "FMM" (K-harmonic F-test motivated by the FMM Fourier expansion; default detector for non-cosinor signals).
<code>K</code>	Integer. Number of harmonics for <code>method = "FMM"</code> (default 2). <code>K=1</code> is equivalent to DCP; <code>K=2</code> captures the 1st and 2nd Fourier harmonics; <code>K=3+</code> for very sharp peaks.
<code>harmonics</code>	Numeric length-2: <code>c(alpha2, alpha3)</code> harmonic coefficients (default <code>c(0, 0)</code>).
<code>verbose</code>	Print progress (default TRUE).
<code>mc.cores</code>	Parallel cores (default 1).
<code>progress</code>	Optional callback function(<code>sample_size_index, n_sample_sizes</code>) invoked after each sample size completes (used by the Shiny app to advance its progress bar). NULL (default) disables the callback.

Details

Simulation-based alternative to the closed-form `CircaPower` formula: works for any design (active or passive) and any pilot parameter distribution.

Value

List with:

<code>marginal_power</code>	Matrix [sample_sizes x nsims]
<code>marginal_FDR</code>	Matrix [sample_sizes x nsims]
<code>marginal_TD</code>	Matrix [sample_sizes x nsims]
<code>marginal_FD</code>	Matrix [sample_sizes x nsims]
<code>marginal_alpha</code>	Matrix [sample_sizes x nsims] – empirical type-I error
<code>pvalues</code>	Array [sample_sizes x ngenes x nsims] – raw p-values

```

r_values_list      List[[sample_size]][[sim]] – per-gene A/sigma
strat_power        Array [sample_sizes x n_strata x nsims]
strat_TD           Array [sample_sizes x n_strata x nsims]
strat_FD           Array [sample_sizes x n_strata x nsims]
strat_n_targets    Array [sample_sizes x n_strata x nsims]
strat_n_nulls      Array [sample_sizes x n_strata x nsims]
strat_FDR          Array [sample_sizes x n_strata x nsims]
strat_alpha        Array [sample_sizes x n_strata x nsims]
r_strata           Stratum breakpoints (from analysis.opts)
strata_labels      Stratum labels
n0_circapower      CircaPower n80 estimate from pilot median r
sample_sizes       Vector of sample sizes
nsims              Number of simulation replicates

```

Examples

```

## Not run:
bio <- estCircadianParam(expr, times)
dopt <- CircadianDesignOptions(sample_sizes = c(20, 40, 80), nsims = 50)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runSimsSingleCohort(bio, dopt, aopt)
rowMeans(res$marginal_power) # mean power at each N

## End(Not run)

```

scp_load_pilot	<i>Load a Bundled SCP Pilot Dataset</i>
----------------	---

Description

Loads one standardized pilot object from the bundled pilot database. Each pilot is a pre-fit `CircadianBioOptions` S3 object (fit via the same cosinor pipeline as [estCircadianParam](#)) and can be passed directly to `runSimsSingleCohort()` or `runDifferentialPower()`.

Usage

```

scp_load_pilot(
  species,
  dataset,
  tissue,
  condition = NULL,
  alpha_pilot = 0.01,
  adjust = c("none", "BH"),
  K = 1,
  paired_sigma = TRUE
)

```

Arguments

species	Character, length 1. One of "human", "baboon", "other_primate", "mouse", "rat".
dataset	Character, length 1. Manifest dataset field (typically GEO accession or short label, e.g. "GTEx", "GSE160521").
tissue	Character, length 1. Tissue or region label as recorded in the manifest (e.g. "Liver", "NAc").
condition	Character, length 1, optional. Condition arm (e.g. "Control", "All"). If NULL and the (species, dataset, tissue) triple identifies more than one row, the function errors and prints the available conditions.
alpha_pilot	Numeric in $(0, \text{cap}]$. Rhythmicity threshold used to re-derive <code>prop_rhythmic</code> and the amplitude/phase/sigma distributions from the pilot's stored per-gene rhythm table. Default 0.01 matches the build-time threshold. Only takes effect for pilots rebuilt with a <code>rhythm_fit</code> table; older pilots ignore it (with a warning) and return their frozen build-time distributions.
adjust	Multiple-testing adjustment applied to the per-gene p-values before thresholding at <code>alpha_pilot</code> : "none" (raw cosinor p, default) or "BH" (Benjamini-Hochberg q-value).
K	Harmonic order of the requested pilot: 1 (single-harmonic cosinor, default) or 2 (two-harmonic). For $K = 2$ the two-harmonic variant of the pilot (<code><file>_2H.rds</code>) is loaded; an error is raised if no such variant has been built for the pilot.
paired_sigma	Logical (default TRUE). Keep the per-gene (A , σ) pairing so the simulator draws amplitude and noise from the same pilot gene, preserving the realistic narrow $\tilde{r} = A/\sigma$ distribution (the marginal-power / Fig 1-2 Panel A convention). FALSE decouples sigma from amplitude, giving the wide $\tilde{r}$ used by the effect-size-stratified panels (Fig 1-2 Panels B/C).

Value

A `CircadianBioOptions` object with empirical distributions of mesor, log-sigma, amplitude, phase, and proportion rhythmic, ready for downstream simulation.

Examples

```
## Not run:
bio <- scp_load_pilot("human", "GTEx", "Liver")
bio <- scp_load_pilot("human", "GTEx", "Liver", alpha_pilot = 0.05)
bio <- scp_load_pilot("human", "GTEx", "Liver", alpha_pilot = 0.1, adjust = "BH")
bio <- scp_load_pilot("human", "GSE160521", "NAc", "Control")

## End(Not run)
```

scp_pilot_search *Search the SCP Pilot Database*

Description

Convenience filter over the pilot manifest. Combines free-text substring matching against dataset, tissue, citation, and accession fields with optional species and tissue-regex filters and an in-vivo vs cell-line restriction. Useful when a user wants to pick a pilot whose design (tissue, sample size, signal strength) matches their planned study.

Usage

```
scp_pilot_search(
  query = NULL,
  species = NULL,
  tissue_pattern = NULL,
  sample_type = "in_vivo"
)
```

Arguments

query	Character, optional. Case-insensitive substring matched against dataset, tissue, accession, citation. NULL matches everything.
species	Character, optional. Restrict to these species.
tissue_pattern	Character, optional. Regular expression matched against tissue (case-insensitive).
sample_type	Character. Default "in_vivo". Use NULL to include cell-line pilots as well.

Value

A data.frame: rows of the manifest passing every active filter.

Examples

```
## Not run:
  scp_pilot_search("liver")
  scp_pilot_search(species = "mouse", tissue_pattern = "brain|cortex")
  scp_pilot_search(sample_type = NULL) # include cell lines

## End(Not run)
```

 scp_pilots

List Bundled SCP Pilot Datasets

Description

Reads the bundled pilot manifest (inst/extdata/pilots/manifest.csv) and returns it as a data.frame. Optionally filters by species and ingestion status so a user can quickly enumerate the pilots they actually have available.

Usage

```
scp_pilots(species = NULL, status = "ingested")
```

Arguments

species	Optional character. One or more of "human", "baboon", "other_primate", "mouse", "rat". NULL (default) returns all species.
status	Character. Filter on the status column. Defaults to "ingested" (datasets actually present on disk). Use NULL or "all" to return every row, including "pending", "verify_failed", "controlled", and "ingest_failed" entries.

Value

A data.frame with one row per (dataset, tissue, condition) pilot. Columns: species, dataset, tissue, condition, n, ngenes, design, assay, sample_type, accession, citation, license, file, status.

Examples

```
## Not run:
  scp_pilots()
  scp_pilots(species = "human")
  scp_pilots(status = "all")

## End(Not run)
```

simCircadianSingleCohort2H

Simulate Single-Cohort Circadian Data Under the Two-Harmonic Cosinor Model

Description

Two-harmonic analog of `simCircadianSingleCohort`. Generates expression from

$$y_{gi} = \mu_g + A_{g,1} \cos(\omega_0(t_i - \phi_{g,1})) + A_{g,2} \cos(2\omega_0(t_i - \phi_{g,2})) + \epsilon_{gi}$$

using the joint pilot tuple $(A_1, \phi_1, A_2, \phi_2, \sigma)$ stored on `bio.opts` by `estCircadianParam2H`. The same index is drawn across all five vectors so the pilot's cross-parameter dependence is preserved.

`r_values` is reported on the 1st-harmonic scale $r_g = A_{g,1}/\sigma_g$ to match the existing r-stratification machinery and the manuscript's r_g notation.

Usage

```
simCircadianSingleCohort2H(bio.opts, cts, seed = NULL)
```

Arguments

<code>bio.opts</code>	A <code>CircadianBioOptions</code> produced by <code>estCircadianParam2H</code> (must carry <code>paired_2h = TRUE</code> , <code>amplitude2</code> , <code>phase2</code>).
<code>cts</code>	Numeric vector of sample collection times in hours; length determines N .
<code>seed</code>	Optional integer random seed.

Value

Named list:

`expr` Gene expression matrix (ngenes x N).

`is_rhythmic` Logical vector.

`r_values` Per-gene 1st-harmonic SNR $A_{g,1}/\sigma_g$.

`A1, phi1, A2, phi2` Per-gene truth vectors (length ngenes; zero for non-rhythmic genes).

See Also

[estCircadianParam2H](#), [simCircadianSingleCohort](#)

plot.SCPDiffResult *Plot an SCPDiffResult object*

Description

Plot an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
plot(x, output_file = NULL, ...)
```

Arguments

x An SCPDiffResult from runDifferentialPower().
output_file Optional PDF path; NULL draws to the active device.
... Ignored.

Value

x, invisibly.

plot.SCPRecommendResult
Plot an SCPRecommendResult object

Description

Plot an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
plot(x, output_file = NULL, ...)
```

Arguments

x An SCPRecommendResult from recommendDesign().
output_file Optional PDF path; NULL draws to the active device.
... Passed to the underlying plot method.

Value

x, invisibly.

```
plot.SCPSingleResult
```

Plot an SCPSingleResult object

Description

Plot an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid().
output_file	Optional PDF path; NULL draws to the active device.
...	Ignored.

Value

x, invisibly.

```
print.CircadianAnalysisOptions
```

Print a CircadianAnalysisOptions object

Description

Print a CircadianAnalysisOptions object

Usage

```
## S3 method for class 'CircadianAnalysisOptions'
print(x, ...)
```

Arguments

x	A CircadianAnalysisOptions object.
...	Ignored.

Value

x, invisibly.

```
print.CircadianBioOptions
```

Print a CircadianBioOptions object

Description

Print a CircadianBioOptions object

Usage

```
## S3 method for class 'CircadianBioOptions'
print(x, ...)
```

Arguments

x	A CircadianBioOptions object.
...	Ignored.

Value

x, invisibly.

```
print.CircadianBootstrapOptions
```

Print a CircadianBootstrapOptions object

Description

Print a CircadianBootstrapOptions object

Usage

```
## S3 method for class 'CircadianBootstrapOptions'
print(x, ...)
```

Arguments

x	A CircadianBootstrapOptions object.
...	Ignored.

Value

x, invisibly.

```
print.CircadianDesignOptions
```

Print a CircadianDesignOptions object

Description

Print a CircadianDesignOptions object

Usage

```
## S3 method for class 'CircadianDesignOptions'
print(x, ...)
```

Arguments

x	A CircadianDesignOptions object.
...	Ignored.

Value

x, invisibly.

```
print.npower
```

Print an npower result

Description

Print an npower result

Usage

```
## S3 method for class 'npower'
print(x, ...)
```

Arguments

x	An npower object (recommended n and its power curve).
...	Ignored.

Value

x, invisibly.

```
print.SCPDiffResult
```

Print an SCPDiffResult object

Description

Print an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
print(x, ...)
```

Arguments

x	An SCPDiffResult from runDifferentialPower().
...	Ignored.

Value

x, invisibly.

```
print.SCPRecommendResult
```

Print an SCPRecommendResult object

Description

Print an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
print(x, ...)
```

Arguments

x	An SCPRecommendResult from recommendDesign().
...	Ignored.

Value

x, invisibly.

```
print.SCPSingleResult
```

Print an SCPSingleResult object

Description

Print an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
print(x, ...)
```

Arguments

<code>x</code>	An SCPSingleResult from <code>runSingleCohortGrid()</code> .
<code>...</code>	Ignored.

Value

`x`, invisibly.

Index

CircadianAnalysisOptions, [1](#)
CircadianBioOptions, [2](#)
CircadianBootstrapOptions, [4](#)
CircadianDesignOptions, [5](#)
circaPowerApproxN80, [6](#)

detect_cosinor, [7](#)
detect_DCP, [7](#)

estCircadianParam, [7](#), [9](#), [11](#), [26](#)
estCircadianParam2H, [9](#), [29](#), [30](#)
estCircadianParamFMM, [9](#)
estCircadianParamTwoGroup, [9](#), [11](#)

launchShiny, [12](#)

makeAdaptiveRStrata, [13](#)

npower, [14](#)

plot.SCPDiffResult, [30](#)
plot.SCPRecommendResult, [30](#)
plot.SCPSingleResult, [31](#)
plotBootstrapDesignGrid, [15](#)
plotDiffPower, [15](#)
plotSingleCohortPower, [17](#)
prepCircadianData, [9](#), [19](#)
print.CircadianAnalysisOptions,
 [31](#)
print.CircadianBioOptions, [32](#)
print.CircadianBootstrapOptions,
 [32](#)
print.CircadianDesignOptions, [33](#)
print.npower, [33](#)
print.SCPDiffResult, [34](#)
print.SCPRecommendResult, [34](#)
print.SCPSingleResult, [35](#)

recommendDesign, [20](#)
runBootstrapDesignGrid, [22](#)
runDifferentialPower, [23](#)
runSimsSingleCohort, [24](#)

scp_load_pilot, [26](#)
scp_pilot_search, [27](#)

scp_pilots, [28](#)
simCircadianSingleCohort, [29](#), [30](#)
simCircadianSingleCohort2H, [11](#), [29](#)